TermTrivia architecture (Phase 3: concurrent server)

Player
(terminal)

types commands

Client process (one per player)

Main thread
command prompt: validates input, sends one line,
then waits for its reply

Reader thread
the only caller of recv(): bytes -> lines -> messages
routes each message by its first line

reply

push

Reply queue
mutex + condition

Terminal output
pushes shown at once

next reply goes back to the main thread

x N clients (20 tested), each with its own connection

TCP request

one line + newline

TCP reply or push

blank-line terminated

Server process

Accept loop (main thread)
accepts, registers the connection,
starts its handler thread,
reaps finished threads

starts

Client handler threads
one per connection (N)
owns a shared_ptr<Connection>
reads lines, parses commands
takes the game lane, applies
the command, writes the effects

lock lane

deliver effects

Connections
socket, receive buffer,
write lock, 200 ms send timeout

look up

Game timer threads
one per game (G), ticks only its game
every 20 ms; pushes QUESTION,
ROUND_RESULT, GAME_END

lock lane

Game lanes (one mutex per game)

Lane: game A
held from state change to last write

Lane: game B
games write in parallel

lock order: lane > routing > state

Routing mutex (short, no socket I/O)
game code -> lane map
connection id -> Connection map

state calls

GameManager (no sockets)
games table, players, scores,
round state, reconnect sessions
state mutex, short operations
returns effects (messages to send)

Logger
timestamped server.log
tokens never logged

Question bank
file read once at startup

Handler N and the timer of game G only ever block each other when they work on the same game; a stalled reader in game A cannot delay game B.

Shutdown and cleanup
Ctrl+C writes to a wake-up pipe; the accept loop stops.
Handlers see the stop flag within 50 ms; timers see it each tick.
Every worker sets a done flag as its last step; finished workers are joined
by the accept loop, and all remaining workers are joined at shutdown
(handlers first, then timers) before state is destroyed.